

VaR Summary Dashboard — How It Works

Loading, API calls, clicks, drilldowns and the query inspector · React 19 · AG Grid Enterprise · Highcharts 12

Companion to the Widget Platform Plan · reflects the var-dashboard-app codebase · Phase 1 (client-side row model, mock services)

API call goes to services · **Cache first** reuses loaded data if fresh ·

No API pure UI work · **Browser only** stored in this browser

Legend used in every table below.

1 The idea in one page

The screen is **not hard-coded**. Services send one **dashboard definition** that lists the filters, the datasets, the widgets and the layout. The app reads it, fetches each dataset once, and hands each widget the rows it needs. Grids and charts share the same data, so a chart never needs its own API.

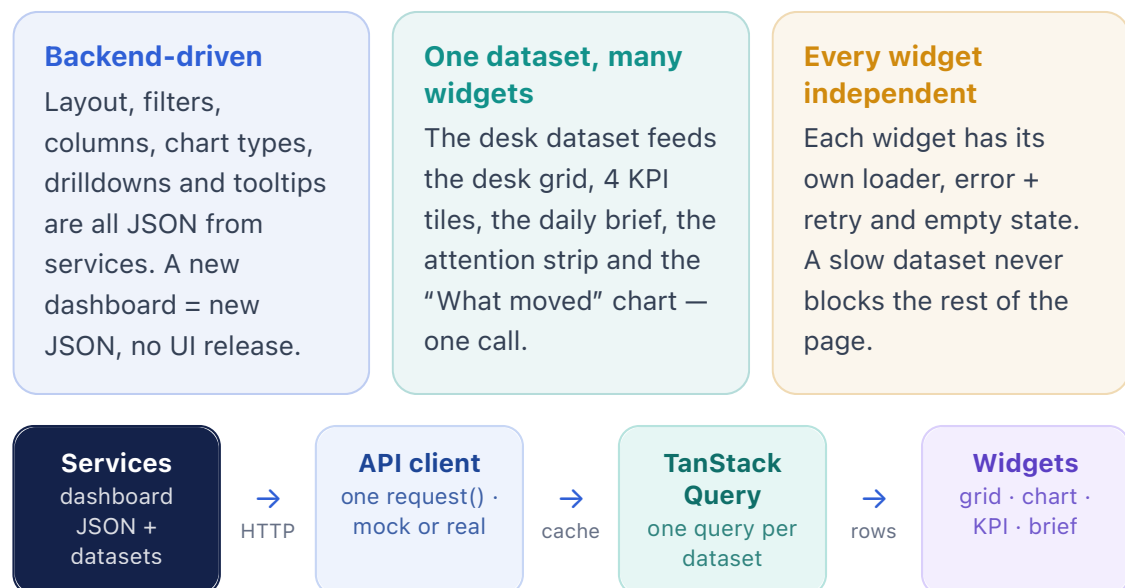


Figure 1. Data path. Filters and selections (Redux) decide the parameters of each query.

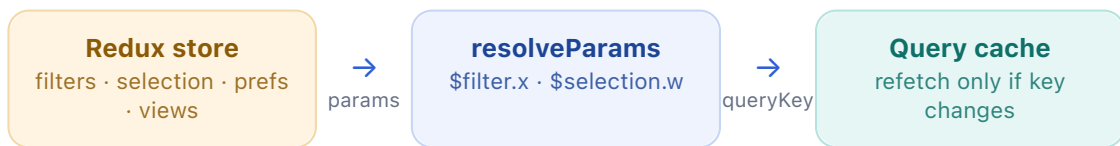


Figure 2. Why only the right widgets reload: the query key contains the resolved parameters.

2 What happens when the page opens

- 1 Sidebar navigation** **API call**
`GET /me/navigation` — the tree (Daily Risk → Equities → VaR Summary...).
Cached for the session.
- 2 Dashboard definition** **API call**
`GET /dashboards/dashboard-1` — filters, 6 dataset references, 16 widgets,
layout rows, tabs, alerts, shortcuts. Cached for the session.
- 3 Filters initialised** **Browser only**
Defaults from the definition, overridden by the default saved view or a shared
link. Stored in Redux.
- 4 Datasets requested in parallel** **API call**
One `GET /datasets/{id}?...` per dataset (5 at start). `dataset-6` is lazy —
it waits for a limit selection only if its binding is required.
- 5 Widgets paint as data lands** **No API**
The layout renders immediately; every widget shows its own soft spinner and
paints the moment its dataset arrives — no waterfall.
- 6 Live polling starts** **API call**
`GET /batches/latest` every 15 s while the tab is visible, to detect a new
risk batch.

First load = 7 small calls (navigation, definition, 5 datasets). After that, clicks
mostly reuse the cache; only filters and desk/limit drilldowns fetch again.

How a component gets loaded

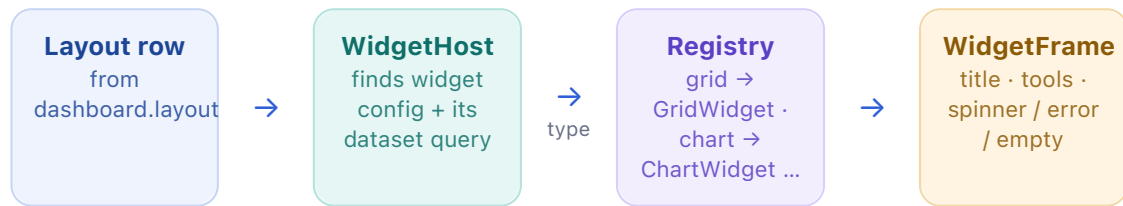


Figure 3. The renderer never knows about “desks” or “limits” — it only knows widget types.

STATUS	WHEN	WHAT THE USER SEES
loading	first request for this dataset	Soft spinner + “Loading...” inside the card
refreshing	new params or new batch; old data kept	Previous numbers stay visible under a frosted spinner — no blank flash
empty	dataset returned 0 rows	Message from the widget config (e.g. “No desks match”)
error	HTTP 5xx / network	Error icon + Retry (refetches only this dataset)
forbidden	HTTP 403	“You don’t have access to this panel”

Table 1. Widget states — the same for every widget type.

3 API catalogue

All calls go through one function, `request()` in `src/api/client.ts`. It switches between mock services and real services with `VITE_USE MOCKS` and `VITE_API_BASE`, times every call for the dev console, and turns errors into widget states.

ENDPOINT	CALLED WHEN	RETURNS	CACHE
GET <code>/me/navigation</code>	App start	Sidebar items (section, parent, route, badge)	Session
GET <code>/dashboards/{id}</code>	Opening a dashboard	Filters, dataset refs + bindings, widgets, layout, tabs, alerts, hotkeys, built-in views	Session
GET <code>/datasets/{id}? params</code>	Start, filter change, drilldown, refresh, Retry	Grid: rowData + columnDefs + totals + meta · Series: rowData + seriesDefs + meta	Per params, 1 min fresh, 10 min kept
GET <code>/batches/latest</code>	Every 15 s (tab visible)	<code>{ batchId, publishedAt }</code>	None

Table 2. Phase 1 endpoints. Saved views, density and column choices stay in the browser in Phase 1; a `/me/views` endpoint is planned for Phase 2.

Which widget uses which dataset

DATASET	CONTENT	PARAMETERS	USED BY
dataset-1	VaR / SVaR by desk (grid)	all filters	Desk grid, 4 KPIs, Daily brief, Attention strip, What moved
dataset-2	VaR trend, 90 business days (series)	filters + <code>deskId?</code>	VaR trend chart, KPI sparklines
dataset-3	Component VaR by FS type (grid)	filters + <code>deskId?</code>	FS type grid
dataset-4	Component VaR over time (series)	filters + <code>deskId?</code>	Stacked FS chart
dataset-5	VaR limits (grid)	COB date	Limits grid
dataset-6	Exposure vs limit (series)	COB + <code>limitId?</code>	Exposure vs limit chart

Table 3. 6 datasets, 16 widgets. ? = optional binding: without a selection the service returns the all-desks / highest-utilisation default.

4 Every click — what changes and what is called

USER ACTION	STATE CHANGE	NETWORK	RESULT ON SCREEN
Change a top filter (Compare, Region, Horizon, Model, Attribute, Measure)	filters[x] in Redux	API call	Every dataset whose params use that filter refetches (key changed). Old numbers stay visible with the spinner, then update with a row flash.
Change COB date	filters.asOfDate	API call	All 6 datasets refetch.
Click a desk row (desk grid)	selection[widget-3] = deskId	API call	datasets 2, 3, 4 refetch with <code>deskId</code> . Trend, FS grid and FS chart show that desk. What moved highlights its bar. Cross-filter chip appears in the top bar.
Click a bar in "What moved"	selection[widget-3] = deskId	API call	Same as clicking the desk row — the chart targets the desk grid.
Click an FS type row	selection[widget-7] = FS code	No API	Stacked FS chart isolates that FS series (client-side, data already loaded).
Click an FS legend item	selection[widget-7]	No API	Same filter as the FS row; chip in the top bar.
Click a limit row	selection[widget-8] = limitId	API call	<code>dataset-6</code> fetched for that limit → Exposure vs limit chart.
Click an attention pill / Daily brief item	selection on its target widget	Depends	Selects the desk (→ API as above), switches to Overview and scrolls to the grid.

Remove a chip / Esc / Clear all	selection cleared	Cache first	Returns to the all-desks view — usually instant from cache.
Apply a saved view (1–9)	filters (+ selection) replaced	Cache first	Refetch only for parameter sets not already cached.
Search in a grid (/)	grid quick filter	No API	Rows filtered in the browser.
Columns · Quick layouts · group chips	column visibility (prefs)	Browser only	Grid re-fits width; if > 6 columns the grid takes the full row.
30D / 60D / 90D	chart range (local)	No API	Series sliced in the browser — the dataset already holds 90 days.
Tabs (Overview / Desks / FS Type / Limits)	ui.tab	No API	Shows a subset of layout rows. Data stays loaded.
Density (gear)	prefs.density	Browser only	Row height, padding and gaps change across the page.
Export grid (CSV / Excel / copy) · chart (PNG / SVG / CSV)	—	No API	Built from the rows already in the grid/chart.
Export page as PDF	—	No API	Captures the whole page in colour (including below the fold) → PDF.
New data banner → Apply update (R)	appliedBatch	API call	All dataset queries invalidated → refetch → changed cells flash.
DB icon on a card	ui.inspector	No API	Opens the query inspector from cached metadata (section 7).

Table 4. Rule of thumb: **filters and desk/limit drilldowns call services; everything else is instant in the browser.**

5 Drilldowns and cross-filters

Widgets never talk to each other. A click **emits** a selection into the store; other widgets **listen** to it, either through a dataset parameter (new API call) or as a client filter (no call). Both are declared in the dashboard JSON.

```
// desk grid emits a selection (its own id)
{ "widgetId": "widget-3", "interactions": { "emits": [{ "on": "rowClick"
}] } }

// "What moved" emits into the desk grid's selection
{ "widgetId": "widget-5", "interactions": { "emits": [{ "on":
"pointClick", "target": "widget-3" }] } }

// dataset bound to that selection → refetch with deskId (optional
binding)
{ "datasetId": "dataset-3", "params": { "asOf": "$filter.asOfDate",
"deskId": "$selection?.widget-3" } }
```

Listing 1. Emit → store → bind. \$selection.w = required (dataset waits), \$selection?.w = optional (default until selected).

SOURCE (CLICK)	SELECTION STORED ON	WHO REACTS	HOW
Desk grid row	widget-3	VaR trend · FS type grid · FS over time · What moved (highlight) · filter chip	dataset params
What moved bar	widget-3	Same as desk row; desk row highlighted	dataset params
FS type row / FS legend	widget-7	Stacked FS chart (series isolated)	client filter
Limits row	widget-8	Exposure vs limit chart	dataset params
Attention pill / brief item	its target	As the target's own click	either

Table 5. Drilldown map. Selected rows show the blue left bar + tint; chips in the navy strip say what is filtered and which panels it affects.

Keyboard: J / K step through desks (same as clicking rows) · Esc clears · C cycles DoD/WoW/MoM · M cycles VaR/SVaR/Both · 1–9 views · S save view · E export · R refresh · / search · ? help · Ctrl K command palette.

6 Response shapes (what services send)

Grids receive **AG Grid-ready** rows and a safe subset of `columnDefs`. Charts receive rows keyed by date plus `seriesDefs`. No colours, no code — only names of renderers, types and tones.


```

GET /datasets/dataset-1?asOf=2026-09-11&compare=DoD&model=MC&measure=Both
...
{
  "datasetId": "dataset-1", "kind": "grid", "rowKey": "id", "version":
  "b4812-1",
  "asOf": "2026-09-11T11:41:00-04:00",
  "columnDefs": [
    { "field": "label", "headerName": "Desk", "pinned": "left",
    "cellRenderer": "labelCell",
      "tooltipComponent": "detailTooltip", "tooltipComponentParams": {
        "kicker": "Desk", "sections": [ ... ] } },
    { "groupId": "g_var", "headerName": "VaR", "children": [
      { "field": "g_var_cob", "headerName": "COB", "type": ["money"],
        "cellRenderer": "smartCell" },
      { "field": "g_var_d", "headerName": "Δ", "type": ["money",
        "change" ] },
      { "field": "g_var_dp", "headerName": "Δ%", "type":
        ["percentChange" ] } ] },
    { "field": "limitUse", "headerName": "Limit use", "type": ["percent",
      "threshold"],
      "styleRules": [{ "when": { "op": "gte", "value": 0.8 }, "tone":
        "warn" } ] }
  ],
  "rowData": [ { "id": "D01", "label": "Derivatives Desk A", "region":
    "NA",
      "g_var_cob": 52400000, "g_var_d": 3300000, "g_var_dp":
        0.0672, "limitUse": 0.845 } ],
  "pinnedBottomRowData": [ { "id": "total", "label": "Total", ... } ],
  "meta": { "rowCount": 16, "batchId": 4812, "queryId": "q-1a3f", "ms":
    184, "cache": "HIT",
    "lineage": ["risk-engine", "var-mart"], "sql": "SELECT ... (non-
    prod only)" }
}

```

Listing 2. Grid dataset. Values in base units (USD); the UI formats to \$MM, shows the exact amount on hover and Δ% as a signed percent.

Services decide

Rows, columns and groups · which columns are hidden by default · style rules (tones) · tooltip sections · totals · entitlements · query metadata.

UI decides

Formatting (\$MM, %, dates) · colours from tones · renderers · selection highlight · search · column picker · density · exports.

7 Query inspector — the DB icon

A developer feature, hidden from normal users. It answers “where did this number come from?” without opening the network tab.

1 Turn on the dev console Browser only

Ctrl + Shift + D (only in development builds, or when enabled by a flag). A DB icon appears on every card header.

2 Click the DB icon on a card No API

Opens “Query · {widget title}”. **No new API call** — it reads the dataset already in the query cache.

3 Read the request No API

The exact `GET /datasets/{id}?...` the widget used, rebuilt from the resolved filters + selections, with a Copy button, and every parameter as a chip.

4 Read the metadata No API

Widget, dataset, query ID, batch, row count, server time (ms), cache HIT/MISS, as-of, version, mock/live source, fetch status and time — from `dataset.meta`.

5 Lineage and SQL No API

Lineage chain (e.g. risk-engine → var-mart → dataset-1). The SQL text is shown **only in development** and only if services send it.

6 Close the dev console Browser only

DB icons disappear and the inspector closes.

Security: the UI never builds or sends SQL. Services should include `meta.sql` in non-production only; production gets query ID, timings, cache status and lineage.

Dev console tabs

TAB	SHOWS
Network	Every request: path, status, ms, size — slow calls highlighted
State	Current filters, selections and preferences (live Redux view)
Renderers	React render time per widget (Profiler) — spot slow widgets
Data checks	Totals match row sums, no negative VaR, batch consistency across datasets
Mocks	Latency slider, force error / empty / forbidden per dataset, publish a new batch

Table 6. Dev console. Not part of the production bundle when disabled.

8 Live data and refresh

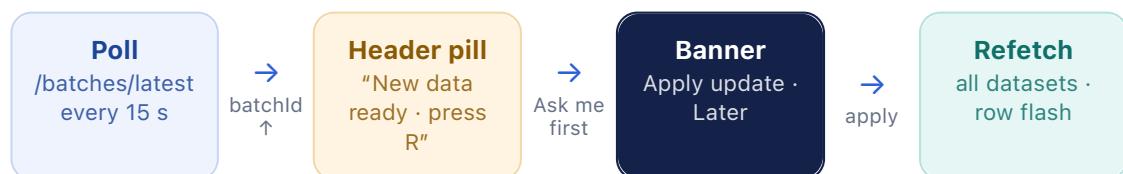


Figure 4. New batch flow. In “Apply automatically” mode it waits until you are not reading a tooltip or using a menu.

Polling pauses when the tab is hidden or the user pauses it. The manual Refresh (R) refetches immediately. A new batch is applied to every widget together, so numbers never mix batches.

9 Browser-only features

FEATURE	HOW IT WORKS	STORED
Saved views	Save the current filters (optionally cross-filters) under a name; 1–9 to switch; star one as default; copy link encodes the view in the URL.	Browser
Columns picker	Quick layouts (Essentials, Full, Analyst, COB only) + group × value matrix; grid re-fits, never scrolls sideways.	Browser
Density	Compact / Comfortable / Spacious for the whole screen.	Browser
Tooltips	Value tooltip shows the exact amount; desk cells show the rich card; icon tooltips — all with a short delay so clicks are not blocked.	—
Exports	Grid CSV / Excel (keeps tones) / copy; chart PNG / SVG / CSV; whole page PDF (one long page or A4).	—
Command palette	Ctrl K: jump to a panel, change a filter, apply a view, export, open help.	—

Table 7. None of these call services in Phase 1.

10 Mock mode vs real services

SETTING	EFFECT
<code>VITE_USE MOCKS=true</code> (default)	In-app mock services (<code>src/mocks</code>). Same contracts, seeded data, simulated latency. No network.
<code>VITE_USE MOCKS=false</code>	Real services at <code>VITE_API_BASE</code> (default <code>/api</code>), cookies included.
Contract checks	In development every response is validated with zod; a wrong field fails loudly. Production trusts the contract for speed.

Table 8. Switching needs no code change — the mocks are the contract handed to services.

11 Questions the team will ask

Why not one API per widget?

Six datasets feed sixteen widgets. Charts reuse grid data, so fewer calls, consistent numbers and one batch id per screen.

Does clicking a desk reload the whole page?

No. Only the three datasets bound to the desk refetch; the others keep their data. The desk grid itself does not reload.

What if one service is slow or down?

Only that card shows the spinner or Retry. Everything else stays usable.

How do we add a new column or chart?

Services add it to the dataset / dashboard JSON using existing types and presets — no UI release. A brand-new renderer or chart preset needs a library change.

How do we add a new dashboard?

A new dashboard definition + datasets and a navigation entry. The same renderer draws it.

Can users see different data?

Yes — entitlements are applied by services per request; a 403 shows the "no access" state on that card.

How big are the responses?

Desk grid ~16 rows, series 90 points × a few series — each well under 100 KB compressed.

Where do colours come from?

Services send tones (positive, negative, warn, breach...); the UI maps them to the design tokens, so the whole app stays consistent and exports keep colours.

12 Where things live in the code

```

src/
  api/client.ts          request() – mock or real, timing, errors
  api/queries.ts        useDashboard · useNavigation · useDatasets ·
useLatestBatch · resolveParams
  contracts/            zod schemas = the API contract
  features/dashboard/   DashboardRenderer · WidgetHost · FilterBar ·
model (filters, selection, ui)
  features/live-refresh/ polling, banner, apply
  features/saved-views/ views bar, save panel, manage menu
  features/dev-tools/   dev console, query inspector, data checks
  features/export-page/ whole-page PDF
  shared/ui/grid/       GridWidget, column types, renderers, tooltip,
columns picker
  shared/ui/charts/     ChartWidget, presets, encode (rows → series)
  shared/ui/widgets/    WidgetFrame, KPI, Daily brief, export menu
  mocks/               dashboard definition, dataset generators, mock
API

```

Listing 3. Code map.